MANUAL DE PROGRAMACIÓN

Taller Integrador de Repaso e Integración

Materia: Laboratorio de Programación (LPR) — 5.º año

Institución: E.E.S.T. N.º 20 “Bangtan Sonyeondan” — Vicente López

Estudiante: Federico Mattia

Archivo principal: src/main.cpp

Lenguaje: C++

Documento técnico de referencia

1. Introducción

El presente manual de programación tiene como finalidad documentar y explicar el desarrollo del programa correspondiente a la Actividad 6: Suite de Taller de Repaso e Integración.

El proyecto está desarrollado en C++ y reúne tres ejercicios que permiten aplicar distintos conceptos fundamentales de la programación: recursividad, arreglos, búsqueda secuencial, direcciones de memoria y punteros.

A lo largo de este documento se describe la estructura del código fuente, el funcionamiento de sus instrucciones, la implementación de las funciones y la relación entre las variables y la memoria.

El manual está dirigido a estudiantes y programadores que necesiten comprender el funcionamiento interno del sistema, realizar modificaciones o utilizar el código como base para futuros ejercicios.

2. Objetivos del desarrollo

Los objetivos principales del programa son:

- Implementar una función recursiva para calcular una suma acumulada.
- Trabajar con arreglos de números enteros.
- Realizar búsquedas secuenciales mediante estructuras repetitivas y condicionales.
- Obtener y mostrar direcciones de memoria.
- Utilizar punteros para modificar variables desde una función.
- Aplicar la modularización mediante la declaración y definición de funciones.
- Comprender el flujo de ejecución de un programa en C++.

3. Estructura general del código

El programa se encuentra organizado en las siguientes partes:

1. Comentario inicial con los datos del proyecto.
2. Inclusión de bibliotecas.
3. Declaración de prototipos de funciones.
4. Función principal `main()`.
5. Desarrollo de la función `sumaRecursiva()`.
6. Desarrollo de la función `intercambiarValores()`.

Esta organización permite separar la función principal de las operaciones específicas, facilitando la lectura, comprensión y mantenimiento del código.

4. Bibliotecas y espacio de nombres

4.1. Inclusión de la biblioteca `iostream`

El programa comienza con la siguiente instrucción:

```
#include <iostream>
```

Esta biblioteca proporciona las herramientas necesarias para realizar operaciones de entrada y salida estándar.

En este proyecto se utiliza principalmente para mostrar información en la consola mediante `cout` y el manipulador `endl`.

4.2. Uso del espacio de nombres `std`

```
using namespace std;
```

Esta instrucción permite utilizar elementos de la biblioteca estándar sin escribir el prefijo `std::` en cada ocasión.

Por ejemplo, gracias a ella se puede escribir `cout` en lugar de `std::cout`.

5. Prototipos de funciones

Antes de la función principal se declaran los prototipos:

```
int sumaRecursiva(int n);
void intercambiarValores(int* ptrA, int* ptrB);
```

Los prototipos informan al compilador sobre el nombre, el tipo de retorno y los parámetros de cada función antes de que aparezca su implementación.

Función `sumaRecursiva`:

- Recibe un número entero mediante el parámetro `n`.
- Devuelve un valor de tipo `int`.
- Se utiliza para calcular una suma acumulada mediante recursividad.

Función `intercambiarValores`:

- Recibe dos punteros a números enteros.
- Su tipo de retorno es `void`, porque no devuelve un valor.
- Modifica los valores de las variables a las que apuntan los punteros.

La declaración previa de estas funciones permite que `main()` pueda invocarlas aunque sus implementaciones estén escritas más adelante.

6. Función principal `main()`

La función principal constituye el punto de entrada del programa. Cuando se ejecuta el programa, las instrucciones de `main()` se procesan secuencialmente.

```
int main() {
    // Instrucciones principales
    return 0;
}
```

En este proyecto, `main()` organiza la ejecución de los tres retos.

6.1. Presentación del programa

Las primeras instrucciones utilizan `cout` para mostrar el título del taller y el nombre del estudiante.

```
cout << "=====================================================" << endl;
cout << "  TALLER INTEGRADOR REPASO - ESTUDIANTE: Federico Mattia " << endl;
cout << "=====================================================" << endl;
```

El operador `<<` permite enviar información a la salida estándar. Por su parte, `endl` finaliza la línea y solicita el vaciado del flujo de salida.

6.2. Organización por retos

Los comentarios separan las distintas partes del programa:

```
// --- RETO 1: RECURSIVIDAD ---
// --- RETO 2: ARRAYS Y BÚSQUEDA SECUENCIAL ---
// --- RETO 3: INTERCAMBIO CON PUNTEROS ---
```

Estos comentarios no afectan la ejecución. Su función es mejorar la legibilidad y facilitar la identificación de cada ejercicio.

7. Reto 1: Recursividad

7.1. Declaración de variables y llamada

Dentro de `main()` se declara la variable `num`:

```
int num = 5;
```

Esta variable almacena el valor que se utilizará para calcular la suma acumulada.

Luego se invoca la función:

```
cout << "Suma acumulada de 1 hasta " << num << ": "
     << sumaRecursiva(num) << endl;
```

La llamada `sumaRecursiva(num)` ejecuta la función y utiliza su resultado como parte del mensaje que se muestra en la consola.

7.2. Implementación de la función recursiva

```
int sumaRecursiva(int n) {
    if (n <= 0) return 0;
    return n + sumaRecursiva(n - 1);
}
```

La función utiliza dos elementos fundamentales.

Caso base:

```
if (n <= 0) return 0;
```

El caso base establece cuándo debe detenerse la recursividad. Si `n` es cero o negativo, la función devuelve cero sin realizar otra llamada recursiva.

Llamada recursiva:

```
return n + sumaRecursiva(n - 1);
```

La función suma el valor actual de `n` con el resultado de una nueva llamada, utilizando un valor menor.

Para `n = 5`, el cálculo se desarrolla de la siguiente manera:

```
sumaRecursiva(5) = 5 + sumaRecursiva(4)
sumaRecursiva(4) = 4 + sumaRecursiva(3)
sumaRecursiva(3) = 3 + sumaRecursiva(2)
sumaRecursiva(2) = 2 + sumaRecursiva(1)
sumaRecursiva(1) = 1 + sumaRecursiva(0)
sumaRecursiva(0) = 0
```

Al resolverse las llamadas pendientes, se obtiene:

5 + 4 + 3 + 2 + 1 = 15

El resultado final es 15.

8. Reto 2: Arreglos y búsqueda secuencial

8.1. Declaración del arreglo

El programa define una constante y un arreglo:

```
const int TAM = 10;
int vectorDatos[TAM] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
```

La constante `TAM` establece que el arreglo contiene diez elementos.

El arreglo `vectorDatos` almacena números enteros en posiciones consecutivas dentro de su región de memoria. Por eso se habla de memoria contigua.

Los índices válidos van del 0 al 9.

8.2. Variables auxiliares

```
int numeroBuscar = 18;
bool hallado = false;
```

La variable `numeroBuscar` contiene el valor que debe encontrarse.

La variable booleana `hallado` indica si el elemento fue localizado. Inicialmente se establece en `false`, porque todavía no se realizó la búsqueda.

8.3. Recorrido del arreglo

```
for (int i = 0; i < TAM; i++) {
    if (vectorDatos[i] == numeroBuscar) {
        cout << "[SISTEMA] Valor " << numeroBuscar
             << " hallado en el indice [" << i << "]" << endl;

        cout << "          Direccion RAM Hexadecimal: "
             << &vectorDatos[i] << endl;

        hallado = true;
        break;
    }
}
```

El ciclo `for` recorre las posiciones del arreglo. La variable `i` comienza en cero y aumenta de uno en uno mientras sea menor que `TAM`.

En cada iteración, la condición:

```
vectorDatos[i] == numeroBuscar
```

compara el elemento actual con el número buscado.

Cuando ambos valores coinciden, se muestra el índice y la dirección de memoria del elemento.

El operador `&`, aplicado a `vectorDatos[i]`, permite obtener la dirección de memoria de esa posición.

Después, la instrucción:

```
hallado = true;
```

registra que el número fue encontrado.

Finalmente:

```
break;
```

interrumpe el ciclo y evita seguir recorriendo el arreglo después de encontrar el valor.

8.4. Comprobación del resultado

```
if (!hallado) {
    cout << "[SISTEMA] El valor " << numeroBuscar
         << " no existe en el arreglo." << endl;
}
```

El operador lógico `!` niega el valor de una expresión booleana.

Por lo tanto, `!hallado` es verdadero si el número no fue encontrado. En ese caso, se muestra un mensaje indicando que el elemento no existe en el arreglo.

Como el número 18 sí está presente, en la ejecución actual se encuentra en el índice 6 y no se muestra el mensaje de error.

8.5. Complejidad de la búsqueda

La búsqueda secuencial puede necesitar revisar todos los elementos del arreglo si el valor buscado no está presente o se encuentra al final.

Por ese motivo, su complejidad temporal en el peor caso es **O(n)**, donde `n` representa la cantidad de elementos.

El uso de `break` permite finalizar anticipadamente la búsqueda cuando se encuentra el valor.

9. Reto 3: Intercambio mediante punteros

9.1. Declaración de las variables

Dentro de `main()` se declaran dos variables enteras:

```
int x = 100, y = 500;
```

La variable `x` comienza con el valor 100 y la variable `y` con el valor 500.

Antes de realizar el intercambio, el programa muestra ambos valores para permitir la comparación con el resultado final.

9.2. Llamada a la función

```
intercambiarValores(&x, &y);
```

En esta llamada se utiliza el operador `&` para pasar las direcciones de memoria de `x` e `y`.

De esta manera, la función recibe referencias indirectas a las variables originales mediante punteros.

9.3. Implementación de la función

```
void intercambiarValores(int* ptrA, int* ptrB) {
    int temporal = *ptrA;
    *ptrA = *ptrB;
    *ptrB = temporal;
}
```

Los parámetros `ptrA` y `ptrB` son punteros a enteros. El asterisco `*` en la declaración indica que se trata de punteros.

Dentro de la función, el operador `*` se utiliza para acceder al valor almacenado en la dirección a la que apunta cada puntero. Esta operación se conoce como desreferenciación.

El intercambio ocurre en tres pasos:

Paso 1. Guardar el primer valor.

```
int temporal = *ptrA;
```

La variable temporal guarda el valor original de `x`, que es 100.

Paso 2. Asignar el segundo valor al primero.

```
*ptrA = *ptrB;
```

El valor de `y`, que es 500, se asigna a `x`.

Paso 3. Recuperar el valor guardado.

```
*ptrB = temporal;
```

El valor 100, guardado en `temporal`, se asigna a `y`.

El resultado final es:

```
Antes:
X = 100
Y = 500

Después:
X = 500
Y = 100
```

Este procedimiento permite modificar directamente las variables originales porque la función trabaja con sus direcciones de memoria.

10. Conceptos fundamentales utilizados

Concepto	Aplicación en el código
Biblioteca `iostream`	Permite mostrar información mediante `cout`.
Variables	Almacenan datos como `num`, `x`, `y` y `numeroBuscar`.
Constantes	`TAM` define el tamaño del arreglo.
Funciones	Separan operaciones específicas del programa.
Recursividad	`sumaRecursiva()` se llama a sí misma.
Caso base	Detiene la recursividad cuando `n <= 0`.
Arreglos	`vectorDatos` almacena diez números enteros.
Ciclo `for`	Recorre las posiciones del arreglo.
Condicionales `if`	Evalúan las condiciones de búsqueda y finalización.
Booleanos	`hallado` registra si se encontró el elemento.
Operador `&`	Obtiene direcciones de memoria en las expresiones utilizadas.
Punteros	`ptrA` y `ptrB` almacenan direcciones de variables enteras.
Desreferenciación `*`	Permite leer y modificar los valores apuntados.
`break`	Finaliza anticipadamente el ciclo de búsqueda.
`return`	Devuelve resultados o finaliza una función.

11. Flujo de ejecución

El orden general de ejecución del programa es el siguiente:

1. El programa ingresa en `main()`.
2. Muestra el encabezado del taller.
3. Inicializa `num` con el valor 5.
4. Llama a `sumaRecursiva()` y muestra el resultado 15.
5. Declara el arreglo, el número buscado y la variable de control.
6. Recorre el arreglo hasta encontrar el número 18.
7. Muestra el índice 6 y la dirección de memoria correspondiente.
8. Inicializa `x` con 100 e `y` con 500.
9. Muestra los valores originales.
10. Llama a `intercambiarValores()` para cambiar ambos valores.
11. Muestra los valores intercambiados.
12. Finaliza con `return 0`.

12. Compilación y mantenimiento

Para compilar el programa con GCC desde la carpeta principal del proyecto, se puede utilizar:

```
g++ -std=c++11 src/main.cpp -o taller
```

El estándar C++11 es suficiente para las características utilizadas en este código.

Si se desea modificar el comportamiento del programa, se deben identificar las variables o funciones correspondientes.

Algunos ejemplos de modificaciones posibles son:

- Cambiar `num` para calcular la suma hasta otro número.
- Modificar `numeroBuscar` para localizar otro elemento.
- Cambiar los valores iniciales de `x` e `y` para probar el intercambio con otros números.
- Modificar los elementos de `vectorDatos` para evaluar diferentes búsquedas.

Después de modificar el código fuente, se debe volver a compilar el programa para que los cambios estén presentes en el ejecutable.

Es importante mantener el caso base de la función recursiva y comprobar que los punteros utilizados tengan direcciones válidas antes de desreferenciarlos.

13. Conclusión

El desarrollo de este programa permite aplicar conceptos esenciales del lenguaje C++ mediante tres ejercicios integradores.

La función recursiva demuestra cómo una operación puede dividirse en llamadas más pequeñas hasta alcanzar una condición de finalización. El ejercicio de búsqueda secuencial permite trabajar con arreglos, índices, ciclos y condiciones. Finalmente, el intercambio mediante punteros muestra cómo acceder a direcciones de memoria y modificar variables desde una función.

La organización del código mediante prototipos, funciones y comentarios facilita su lectura y mantenimiento. Además, el estudio de su funcionamiento interno proporciona una base para desarrollar programas más complejos y comprender mejor la relación entre las instrucciones de C++ y la memoria del sistema.